Progresol

Guía de despliegue y pendientes

Cómo publicar el backend y el frontend de MIGAL SmartOps, qué supuestos faltan validar y qué resta para completar el proyecto.

MIGAL SMARTOPS

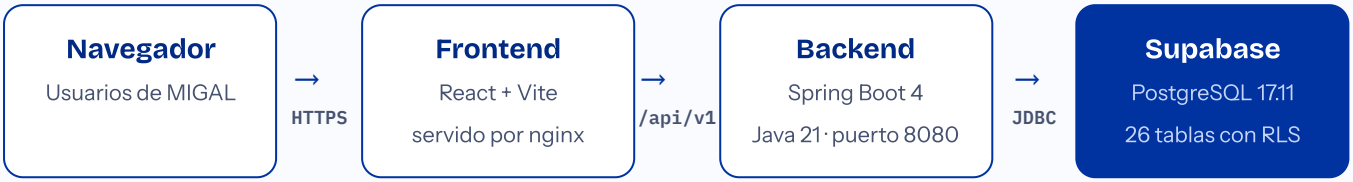
Documento para el equipo de desarrollo

Estado al 11 de noviembre de 2026 · Rama `fix/frontend-revision-visual`

PANORAMA

Qué se despliega y cómo se conecta

El sistema consta de tres piezas. La base de datos ya está en Supabase con el esquema aplicado; faltan los empaquetados de backend y frontend y la conexión entre ambos en un entorno publicado.



Estado actual por componente

Componente	Estado	Detalle
Base de datos	LISTA	Migraciones V1 a V15 aplicadas en Supabase mediante Flyway. Todas las tablas tienen RLS habilitado sin políticas.
Backend	PARCIAL	Funciona localmente contra Supabase. No existe `Dockerfile` ni configuración de CORS; el OCR y el agente de IA usan proveedores simulados.
Frontend	PARCIAL	Tiene `Dockerfile` y `nginx.conf`, pero nginx no reenvía `/api` al backend y la compilación de producción usa datos simulados salvo que se indique lo contrario.
Datos reales	PENDIENTE	La base solo contiene las categorías sembradas por la migración V10. No hay productos, clientes, proveedores ni stock reales.

PRINCIPIO DE DISEÑO (ADR-007)
El backend es el único cliente de la base de datos. El frontend no usa la API ni las claves de Supabase, y las migraciones solo se aplican mediante Flyway para conservar un único historial.

Requisitos previos

Herramientas

Java 21 (se usa el wrapper `mvnw`), Node.js 22.22 o superior, npm y Git. Docker es necesario para las pruebas del backend (Testcontainers) y para construir imágenes.

Accesos

Proyecto `migal-smartops` de Supabase (región us-east-1) y la contraseña de su base de datos, entregada por el responsable por un canal privado.

PASO 1

Base de datos en Supabase

El esquema ya fue aplicado. Esta sección sirve para reproducir la conexión en otro equipo y para futuras migraciones.

1. **Obtener la cadena de conexión.** En Supabase, botón *Connect* → pestaña *Direct*. Usar *Session pooler* (puerto 5432) si la red no admite IPv6. No usar el modo *Transaction* (puerto 6543): interfiere con los bloqueos de sesión que requiere el inventario.

2. **Crear el archivo `.env` en la raíz del repositorio** (copiar `.env.example`). El archivo está excluido de Git y el backend lo lee al iniciar.

```
SPRING_DATASOURCE_URL=jdbc:postgresql://<host>:5432/postgres?sslmode=require
SPRING_DATASOURCE_USERNAME=postgres
SPRING_DATASOURCE_PASSWORD=<contraseña de la base>
MIGAL_JWT_SECRETO=<mínimo 32 caracteres>
MIGAL_ADMIN_USERNAME=admin
MIGAL_ADMIN_CLAVE=<mínimo 8 caracteres, letras y números>
```

3. **Iniciar el backend.** Flyway aplica automáticamente las migraciones pendientes al arrancar. Una migración nueva solo requiere agregar el archivo `V16__descripcion.sql` en `database/migraciones/` y reiniciar.

4. **Verificar.** En el *Table Editor* deben figurar las tablas con el indicador RLS y `flyway_schema_history` con las versiones 1 a 15.

PRIMERA EJECUCIÓN (YA REALIZADA)

El esquema `public` de Supabase incluye una función propia de la plataforma, por lo que Flyway exigió una línea base. Se resolvió con `spring.flyway.baseline-on-migrate=true` y `spring.flyway.baseline-version=0`. Estas dos líneas ya no son necesarias y deben retirarse del `.env`.

SEGURIDAD DE LA CONTRASEÑA

El registro de Hibernate imprime la URL de conexión con la contraseña en texto plano. La contraseña de la base apareció en la consola y en una conversación, por lo que debe rotarse: *Project Settings* → *Database* → *Reset database password*. Evitar compartir registros del backend sin revisarlos.

Reglas que conviene recordar

Regla	Motivo
No modificar una migración ya integrada	Flyway valida la suma de comprobación y falla ante cualquier cambio. Las correcciones se hacen con una migración nueva.
No usar el editor SQL de Supabase para cambiar el esquema	Generaría un segundo historial y divergencias entre entornos (ADR-007).
No cargar `demo.sql` en Supabase	Solo está pensado para el entorno local de Docker y no marca las filas de prueba, por lo que no se podría retirar de forma limpia.

PASO 2

Backend Spring Boot

Se puede ejecutar de forma local (verificado) o en un contenedor (propuesta que aún no existe en el repositorio).

Opción A. Ejecución local VERIFICADA

```
# desde la carpeta backend/
.\mvnw.cmd spring-boot:run
```

Si aparece *Port 8080 was already in use*, hay una instancia anterior en ejecución. Identificar el proceso con `netstat -ano | findstr :8080` y cerrarlo con `Stop-Process -Id <PID>`.

Opción B. Contenedor POR CREAR

La arquitectura exige contenedores (RNF-04), pero el backend no tiene `Dockerfile`. Propuesta de partida, aún no validada:

```
# backend/Dockerfile
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY . .
RUN ./mvnw -B -ntp -DskipTests package
FROM eclipse-temurin:21-jre
COPY --from=build /app/target/*.jar /app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","/app.jar"]
```

MIGRACIONES EN EL CONTENEDOR

El backend busca las migraciones en `filesystem:../database/migraciones`. Dentro de una imagen esa ruta no existe: se debe copiar la carpeta a la imagen y definir `FLYWAY_LOCATIONS` con la nueva ruta.

Variables de entorno del backend

Variable	Obligatoria	Descripción
`SPRING_DATASOURCE_*`	Sí	Conexión a Supabase. Por defecto apunta a PostgreSQL local.
`MIGAL_JWT_SECRETO`	Sí	Clave de firma HS256, mínimo 32 caracteres. Los tokens duran 8 horas.
`MIGAL_ADMIN_USERNAME / CLAVE`	Primer inicio	Crean el primer Administrador si no existe ninguno. Cambiar la clave tras el primer ingreso.
`MIGAL_OCR_PROVEEDOR` `MIGAL_AGENTE_PROVEEDOR`	No	*simulado* (por defecto) o *no-configurado*. No hay proveedor real de OCR ni de modelo de lenguaje.
`MIGAL_DOCUMENTOS_DIRECTORIO`	No	Carpeta de los archivos cargados. En un contenedor requiere un volumen persistente.
`FLYWAY_LOCATIONS`	En contenedor	Ruta de las migraciones.

PASO 3

Frontend React

El frontend llama siempre a rutas relativas (`/api/v1`). En desarrollo, Vite reenvía esas rutas al puerto 8080; en producción debe hacerlo el servidor web.

Desarrollo local VERIFICADO

```
# desde la carpeta frontend/
npm ci
npm run dev        # http://localhost:5173, API real en el puerto 8080
```

Compilación de producción REQUIERE AJUSTES

DATOS SIMULADOS POR DEFECTO

El cliente HTTP usa la API real solo en modo desarrollo. En una compilación de producción la variable `VITE_API_REAL` debe ser `true` al construir; de lo contrario la aplicación mostrará datos simulados.

```
VITE_API_REAL=true npm run build      # genera frontend/dist
# en PowerShell:
$env:VITE_API_REAL="true"; npm run build
```

En el `Dockerfile` actual esto implica agregar, antes de `RUN npm run build` , la línea `ENV VITE_API_REAL=true` .

Reenvío de `/api` en nginx POR AGREGAR

El `nginx.conf` actual solo sirve archivos estáticos. Se necesita un bloque como el siguiente, donde `backend` es el nombre del servicio o el host del backend:

```
location /api/ {
    proxy_pass http://backend:8080;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    client_max_body_size 11m;   # los documentos admiten hasta 10 MB
}
```

POR QUÉ NO HACE FALTA CORS

Si navegador, frontend y `/api` comparten el mismo origen, no se requiere CORS. El backend no tiene configuración de CORS; si se aloja el frontend en otro dominio, habrá que agregarla.

Alternativa con hosting de sitios estáticos

También es posible publicar `dist` en un servicio de hosting estático con una regla de reescritura de `/api/*` hacia el backend y de cualquier otra ruta hacia `index.html` (la aplicación usa rutas del lado del cliente). La elección de proveedor de hosting para backend y frontend **no está definida** y es una decisión pendiente del equipo.

PASO 4

Verificación posterior al despliegue

Lista de comprobación para confirmar que las tres piezas se comunican correctamente.

- [ ] El backend arranca sin errores y el registro muestra *Schema "public" is up to date*.
- [ ] La consulta `/actuator/health` responde estado *UP* (verificar si la ruta es pública con la seguridad configurada).
- [ ] El inicio de sesión con el Administrador inicial funciona desde el navegador y el token se envía en las solicitudes siguientes.
- [ ] El Administrador ve todos los módulos; un usuario de cada rol (vendedor, almacenero, reponedor) ve solo los suyos.
- [ ] El listado de categorías muestra las 8 categorías principales y sus subcategorías.
- [ ] Se puede crear un producto, un almacén, un cliente y un proveedor, y las filas aparecen en Supabase.
- [ ] Un movimiento de inventario genera una fila en el Kardex y actualiza la cantidad física.
- [ ] El flujo completo cotización → validación → venta → reserva → despacho se ejecuta sin errores.
- [ ] La carga de un documento de hasta 10 MB funciona a través de nginx (límite del cuerpo de solicitud).
- [ ] La API pública de Supabase no devuelve datos con la clave anónima (RLS sin políticas).
- [ ] Las contraseñas y la clave JWT no aparecen en ningún registro ni en el repositorio.

Problemas frecuentes

Síntoma	Causa y solución
Flyway: *Found non-empty schema(s) "public" but no schema history table*	El esquema de Supabase no está vacío. Usar la línea base con versión 0 solo en la primera ejecución.
Port 8080 was already in use	Instancia anterior en ejecución. Cerrar el proceso que ocupa el puerto.
La pantalla muestra datos de ejemplo	Compilación sin `VITE_API_REAL=true`.
Error 404 en `/api/...` desde el sitio publicado	nginx o el hosting no reenvían `/api` al backend.
No conecta con `db.<ref>.supabase.co`	La conexión directa usa IPv6 en el plan gratuito. Usar el *Session pooler*.
Las pruebas del backend no inician	Testcontainers requiere Docker en ejecución.

Flujo de trabajo del equipo

Cada cambio entra por pull request hacia `main`, con commits en formato Conventional Commits. Los flujos *verificar-backend* y *verificar-frontend* ejecutan compilación y pruebas. **No existe todavía un flujo de despliegue automático.**

VALIDACIÓN

Supuestos por validar

Decisiones que el código ya asume pero que aún no cuentan con confirmación del docente, del cliente o del equipo.

Supuesto	Situación actual	Quién valida y qué ocurre si cambia
Reserva parcial sin entrega parcial	El ADR-003 reserva lo disponible y registra el resto como pendiente; el despacho es único y completo.	Docente (observaciones de PC2). Si se rechaza, se redacta un ADR que reemplaza la decisión 3 y cambian reservas y despacho.
Viabilidad de abastecimiento por categoría	La migración V10 incluye `aprobacion_automatica` por categoría, pero los requerimientos no la regulan.	Equipo y docente. Incorporar la regla a los requerimientos y confirmar qué categorías son automáticas.
Acceso total del Administrador	Implementado: ve todos los módulos (PR 113).	Cliente. Si es parcial, ajustar permisos en backend y frontend.
Nivel de stock sobre stock libre	Las alertas se calculan sobre el stock libre, no sobre el físico.	Cliente. Afecta alertas de reposición y vistas de inventario.
Umbrales de coincidencia OCR	0.85 (alta) y 0.60 (baja), valores de ejemplo.	Equipo, con documentos reales de MIGAL. Configurables por variable de entorno.
Moneda única	ADR-005 asume una sola moneda, por confirmar.	Cliente. Si hubiera dos monedas, cambia el modelo de precios.
Prioridad entre ubicaciones al reservar	Se define en el backend (tienda y almacenes).	Cliente. Determina de dónde sale primero la mercadería.
Nombres de estados	Definidos en minúsculas sin tildes en la base de datos.	Equipo. Confirmar el texto visible en la interfaz.
Duración de sesión y retención de idempotencia	Token de 8 horas; claves de idempotencia con retención de 24 horas.	Equipo. Valores de ADR-009 y ADR-010.
Plan de Supabase frente a RNF-11 y RNF-12	El plan gratuito pausa proyectos inactivos y no ofrece recuperación a un punto en el tiempo.	Equipo. Verificar disponibilidad y respaldo antes de la puesta en producción.
Proveedores reales de OCR y modelo de lenguaje	No definidos. El microservicio Python de la arquitectura no existe en el repositorio.	Equipo y docente. Definir alcance: simulado para la demostración o integración real.

CÓMO REGISTRAR LA CONFIRMACIÓN

Cada validación debe quedar en la tarea de Notion y, si cambia una regla, en un ADR nuevo o en la actualización del existente, junto con los diagramas y el diccionario de datos.

DATOS

Cargar los datos reales en Supabase

Supabase debe contener solo datos reales. El entorno local de Docker conserva los datos de demostración para probar pantallas.

Orden recomendado de carga

#	Datos	Cómo cargarlos hoy	Observaciones
1	**Categorías**	Ya sembradas (V10)	8 categorías principales y subcategorías. Revisar que coincidan con el catálogo real de MIGAL.
2	**Ubicaciones**	API `/almacenes` o pantalla	Tienda principal y dos almacenes.
3	**Productos**	API `/productos` o pantalla	SKU único, unidad, precio referencial, stock mínimo y categoría.
4	**Proveedores y catálogo**	API `/proveedores` y `/catalogo`	RUC único de 11 dígitos; precio de compra por producto.
5	**Clientes**	API `/clientes` o pantalla	Incluir el cliente de mostrador (documento opcional).
6	**Usuarios**	Pantalla de usuarios	Un usuario por persona y rol; contraseñas individuales.
7	**Stock inicial**	Movimiento de ajuste o entrada	Nunca insertar directo en `inventario`: debe registrarse en el Kardex con motivo para que el saldo coincida con el historial.

POR QUÉ NO SE PUEDE "REEMPLAZAR" DESPUÉS

El Kardex es de solo anexión y las claves foráneas son `RESTRICT`. Una vez que existen movimientos, no se pueden borrar productos ni ubicaciones. Por eso conviene cargar datos reales desde el inicio y no mezclar datos de prueba en Supabase.

Qué falta construir

Importador desde Excel o CSV

No existe. Se propone un script que lea plantillas de productos, clientes y proveedores y llame a la API del backend, de modo que se apliquen las mismas validaciones que en la interfaz. El stock inicial se registraría como movimientos de ajuste con un motivo común, por ejemplo "Inventario inicial".

Insumos de MIGAL

Listado de productos con SKU, unidad, precio y mínimo; stock contado por ubicación; clientes y proveedores con sus documentos; y las personas que usarán el sistema con su rol.

HOJA DE RUTA

Qué falta para completar el proyecto

Resumen priorizado. Las marcas verdes indican lo ya realizado.

Hecho

- [x] Esquema de base de datos (15 migraciones), DER y diccionario de datos alineados.
- [x] Backend con los módulos de seguridad, catálogo, inventario, comercial, despacho, abastecimiento, OCR y agente.
- [x] Frontend conectado a la API real y esquema aplicado en Supabase con RLS.

Prioridad alta: necesario para publicar

Tarea	Detalle
Rotar la contraseña de la base de datos	Expuesta en consola y conversación.
`Dockerfile` del backend y `compose` completo	Backend, frontend y variables externas a la imagen (RNF-04).
Reenvío de `/api` en nginx y `VITE_API_REAL=true`	Sin esto el sitio publicado no usa el backend.
Hosting de backend y frontend	Elegir proveedor y dominio; definir HTTPS.
Roles de base de datos (migración y aplicación)	El rol de la aplicación sin `UPDATE` ni `DELETE` sobre el Kardex (ADR-007, punto 6). Hoy se usa el usuario `postgres`.
Carga de datos reales	Insumos de MIGAL, importador y stock inicial por Kardex.
Almacenamiento persistente de documentos	Volumen en el contenedor o decisión de otro almacenamiento (el ADR-007 limita Supabase a PostgreSQL).

HOJA DE RUTA

Qué falta (continuación)

Prioridad media

Tarea	Detalle
Respaldo y recuperación (RNF-12)	ADR de despliegue pendiente: frecuencia, retención y pruebas de restauración.
Proyectos separados por entorno	Hoy un único proyecto de Supabase sirve como entorno compartido.
Despliegue automático (CI/CD)	Hoy solo existen verificaciones de pull request.
Limpieza de claves de idempotencia	El ADR-010 prevé una limpieza periódica; no se encontró una tarea programada en el backend.
Pruebas de rendimiento (RNF-06)	Consultas de inventario, compras, cotizaciones y ventas en 3 segundos o menos.
Confirmar los supuestos de la página anterior	Especialmente reserva parcial y viabilidad por categoría.

Según el alcance acordado

Tarea	Detalle
OCR y modelo de lenguaje reales	Hoy simulados. Definir si la entrega los incluye y, de ser así, el proveedor y las credenciales.
Diagramas de secuencia y actividades	CU-05, CU-06, CU-09, CU-10 y CU-13 deben actualizarse según el ADR-003.
Capacitación y manual de usuario	Para las personas de MIGAL que usarán el sistema.

Entorno local con datos de demostración

```
docker compose up -d
docker exec -i migal-postgres psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" \
  -v ON_ERROR_STOP=1 < database/datos-demostracion/demo.sql
```

Requiere que el backend haya creado antes el Administrador inicial. Es idempotente.